上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

2026年春季学期

编译原理
Principles of Compilers

第三章：词法分析（2）

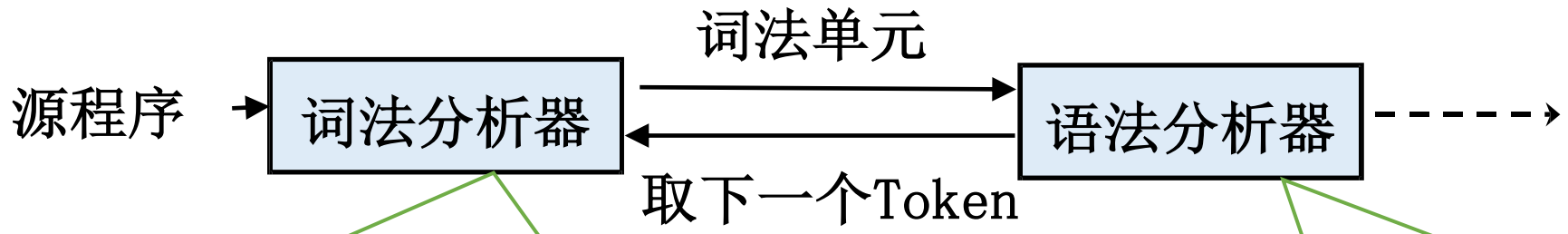
赵杰茹

上海交通大学计算机学院

内容纲要

- 词法分析器的作用
- 词法单元的规约
- **词法单元的识别**
- 词法分析器生成工具LEX
- 有穷自动机 (Finite Automata)
  - NFA，DFA
- 正则表达式到有穷自动机
- 词法分析器生成工具的设计

如何识别词法单元?



词法单元的模式

digit	→	[0-9]
digits	→	*digit*$^+$
number	→	*digits* (. *digits*)? (E [+-]? *digits*)?
letter	→	[A-Za-z]
id	→	*letter* (*letter* | *digit*)*
if	→	if
then	→	then
else	→	else
relop	→	< | > | <= | >= | = | <>

分支语句的文法

stmt	→	**if** *expr* **then** *stmt*
	|	**if** *expr* **then** *stmt* **else** *stmt*
	|	ϵ
expr	→	*term* **relop** *term*
	|	*term*
term	→	**id**
	|	**number**

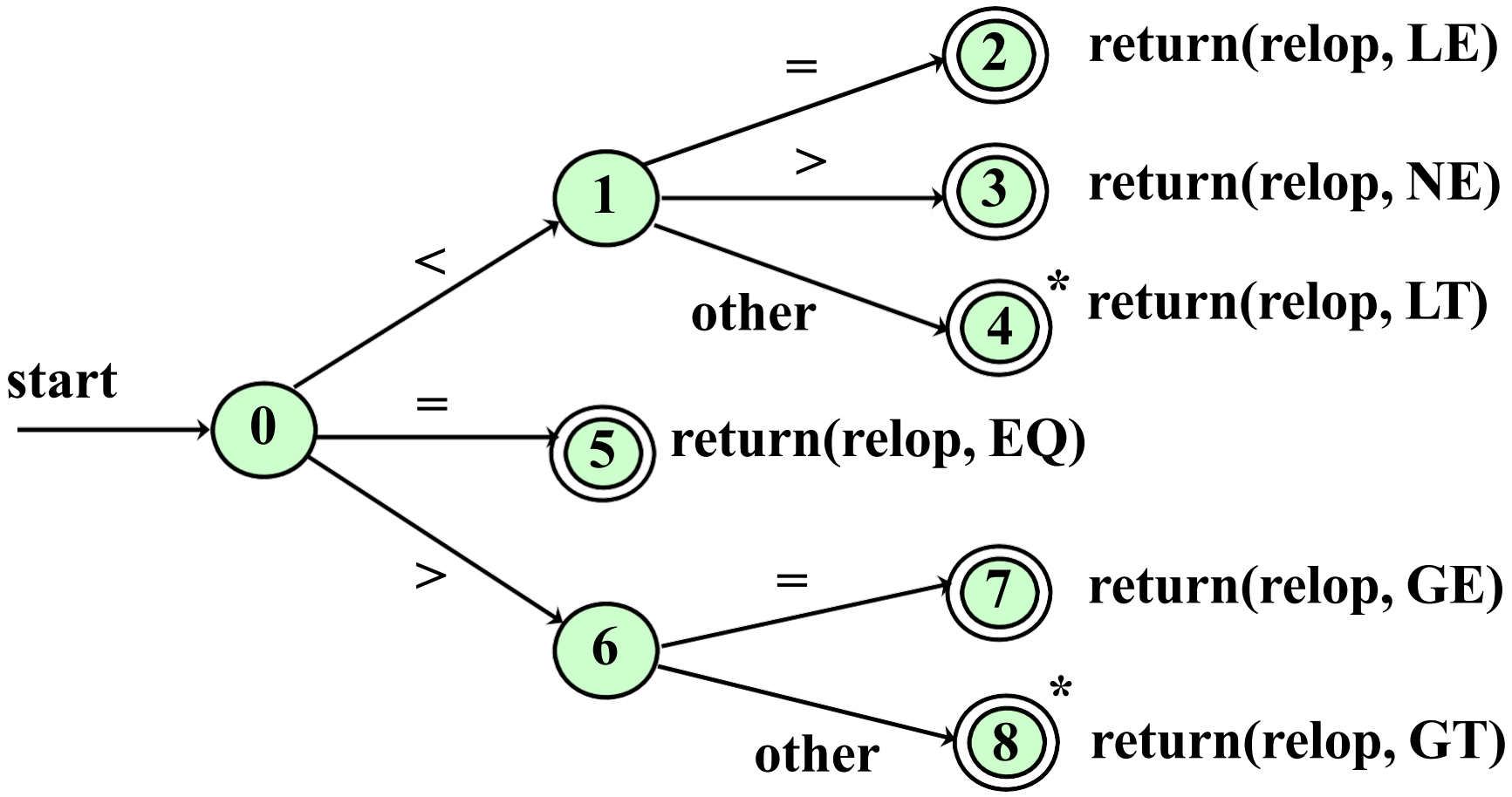
- 如何识别输入字符流中的每一个词素?

状态转换图

- 状态转换图（transition diagram）
  - 由模式转换成的具有特定风格的**流图**
  - 状态（state）：表示在识别词素时可能出现的情况
    - 状态看作是对已处理字符的总结
    - 某些状态为**接受状态**或**最终状态**，表明已找到词素，用双层圈表示
    - 在**接受状态上加n个***表示最后读入的n个字符不在识别出的词素中
    - 有一个状态为**开始状态**，也称**初始状态**，由标号为“开始（start）”的边表示
  - 边（edge）：表示从一个状态指向另一个状态；边的标号是一/多个符号
    - 当前状态为s，下一个输入符号为a，就沿着一条从s离开且标号为a的边到达下一个状态

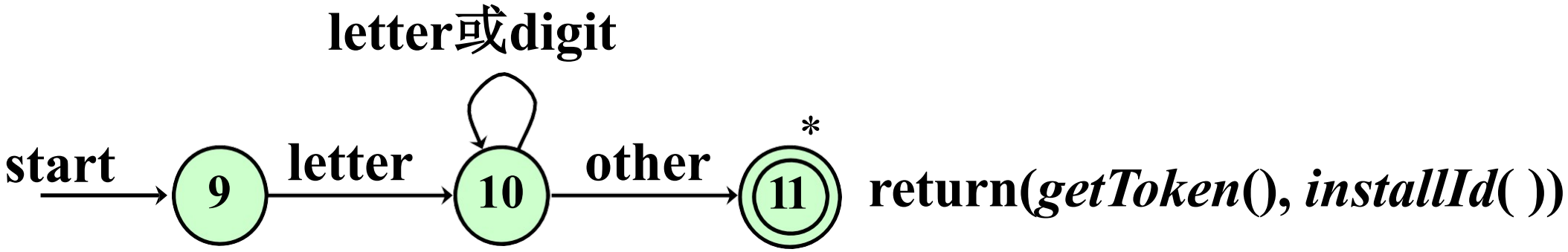
识别比较运算符的状态转换图

relop → < | > | <= | >= | = | <>



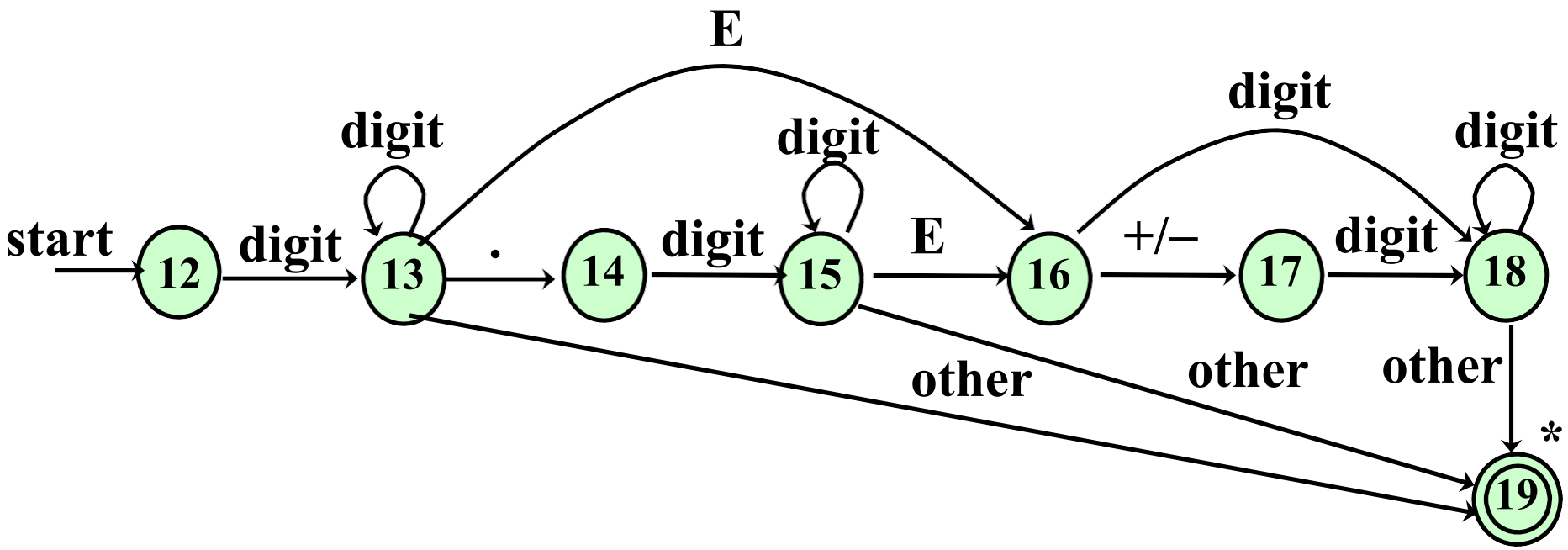
识别标识符和保留字的状态转换图

id	→	*letter* (*letter* | *digit*)*
if	→	`if`
then	→	`then`
else	→	`else`



识别无符号数字的状态转换图

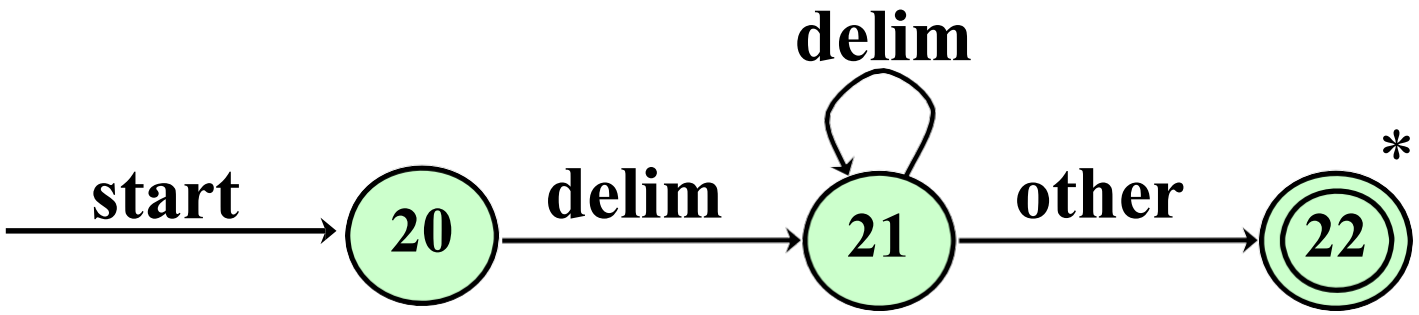
$$
\begin{array}{rcl}
digit & \rightarrow & [0\text{-}9] \\
digits & \rightarrow & digit^{+} \\
number & \rightarrow & digits\ (\,.\ \ digits)?\ \ (\ \mathtt{E}\ [+\text{-}]?\ \ digits\ )?
\end{array}
$$



识别空白符的状态转换图

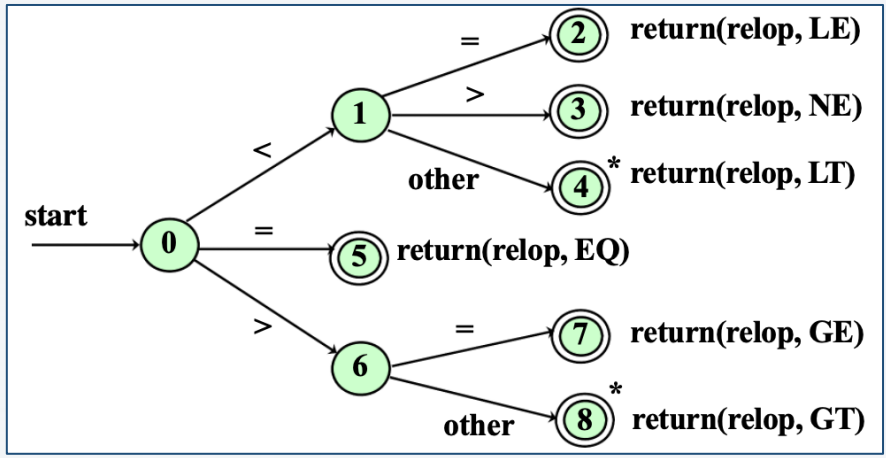
$delim \rightarrow$ blank | tab | newline

$ws \rightarrow delim^{+}$



手动编写词法分析程序

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                  or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```



词法单元识别过程

- 对于输入字符串，如何连续识别多个不同的词法单元?

```
\tif (i == j)\n\t\tz = 0;\n\telse\n\t\tz = 1;
```

- 识别过程:
  1) 分别为每一类词法单元写出正则表达式 R_i
  2) 构造一个正则表达式 R 来匹配所有的词法单元

$$R = R_1 \mid R_2 \mid \cdots \mid R_k$$

  3) 设输入为 $x_1x_2\cdots x_n$，对$1 \leqslant i \leqslant n$，检查是否满足 $x_1\cdots x_i \in L(R)$
  4) 如果匹配成功，则存在j，使得 $x_1\cdots x_i \in L(R_j)$
  5) 把$x_1\cdots x_i$从输入中移走，继续执行步骤3)

匹配过程中需要解决的问题

- 选择哪个正则表达式的状态转换图来匹配?
  - 有可能多个正则表达式都可以匹配：标识符 vs 保留字

  💡 **排在前面的正则表达式优先匹配**

- 如何确定匹配的长度?
  - 有可能多个前缀都可以产生匹配：then vs thenext

  💡 **取最长的和某个模式匹配的串**

- 如果所有正则表达式都不能匹配怎么办?
  - 需要报错

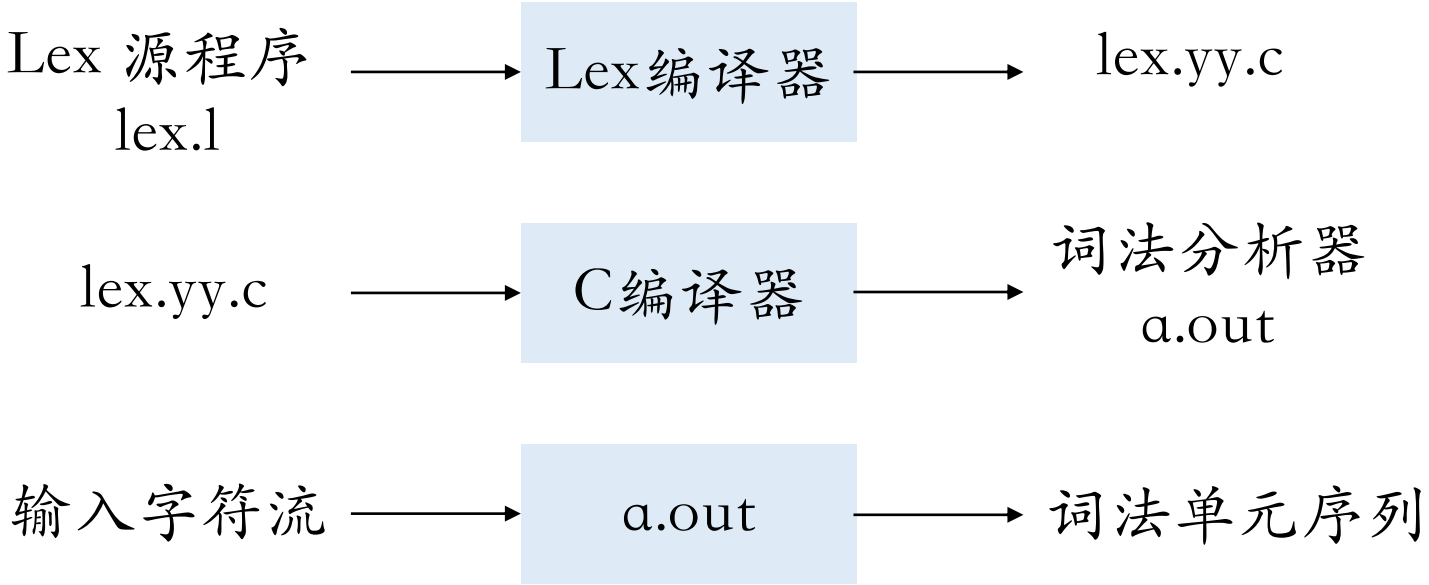
  💡 **构造一个ERROR正则表达式，**放到所有正则表达式后，以报告错误

内容纲要

- 词法分析器的作用
- 词法单元的规约
- 词法单元的识别
- **词法分析器生成工具LEX**
- 有穷自动机 (Finite Automata)
  - NFA，DFA
- 正则表达式到有穷自动机
- 词法分析器生成工具的设计

Lex简介

- Lex是一个词法分析程序的**自动构造**工具
  - 原理：根据给定的正则表达式，转换成状态转换图，并自动生成相应的词法分析程序。



Lex源程序的结构

- 一个Lex程序主要包含三部分，用%%隔开：
  - 声明部分
    - 变量
    - 明示常量（manifest constant）：表示一个常数的标识符，如NE，GE
    - 正则定义
  - 转换规则
    - 形式：模式 {动作}
    - 每个模式是一个正则表达式，相应的动作为拟执行的代码片段
  - 辅助函数
    - 各动作需要使用的辅助函数
    - 也可以将辅助函数单独编译，再与词法分析器代码一起装载

Lex程序示例

声明部分：

```
%{
    /* definitions of manifest constants
    LT, LE, EQ, NE, GT, GE,
    IF, THEN, ELSE, ID, NUMBER, RELOP */
%}

/* regular definitions */
delim       [ \t\n]
ws          {delim}+
letter      [A-Za-z]
digit       [0-9]
id          {letter}({letter}|{digit})*
number      {digit}+(\.{digit}+)?(E[+-]?{digit}+)?

%%
```

明示常量

正则定义

Lex程序示例（续）

转换规则：模式{动作}

```
{ws}        {/* no action and no return */}
if          {return(IF);}
then        {return(THEN);}
else        {return(ELSE);}
{id}        {yylval = (int) installID(); return(ID);}
{number}    {yylval = (int) installNum(); return(NUMBER);}
"<"         {yylval = LT; return(RELOP);}
"<="        {yylval = LE; return(RELOP);}
"="         {yylval = EQ; return(RELOP);}
"<>"        {yylval = NE; return(RELOP);}
">"         {yylval = GT; return(RELOP);}
">="        {yylval = GE; return(RELOP);}

%%
```

返回词法单元名

把属性值存放于全局变量yylval中

辅助函数：

```
int installID() {/* function to install the lexeme, whose
                    first character is pointed to by yytext,
                    and whose length is yyleng, into the
                    symbol table and return a pointer
                    thereto */
}
```

向符号表中添加词素对应的条目，并返回指针

```
int installNum() {/* similar to installID, but puts numer-
                     ical constants into a separate table */
}
```

把数字常量添加到另一个单独的表中

由Lex创建的词法分析器如何工作?

- 工作方式:
  - 由Lex生成的词法分析器作为一个函数被语法分析器调用
  - 调用时，词法分析器从余下的输入中逐个读取字符
  - 发现**最长的**与某个模式匹配的的前缀时，**执行相关动作**
    - 该动作进行相关处理，再将控制返回给语法分析器
    - 如果不返回（比如空白符），词法分析器将继续寻找其他词素
  - 词法分析器只向语法分析器返回词法单元名，将属性值存入与语法分析器共享的全局变量yylval中，来传递相应词素的附加信息

Lex中的冲突解决

- 冲突：当输入的多个前缀与一个或多个模式匹配

- Lex的解决办法：
  - then vs thenext， <= vs <

  💡如果多个前缀可能匹配，**总是选择最长的前缀**
  - then：保留字 vs 标识符

  💡如果前缀和多个模式匹配时，**总是选择列在前面的模式**

内容纲要

- 词法分析器的作用
- 词法单元的规约
- 词法单元的识别
- 词法分析器生成工具LEX
- 有穷自动机 (Finite Automata)
  - NFA，DFA
- 正则表达式到有穷自动机
- 词法分析器生成工具的设计

有穷自动机（Finite Automata）

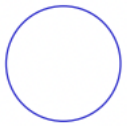
- 如何理解？
  - 有穷自动机是一种数学模型，状态转换图是可以用来描述有穷自动机的直观图示
  - 正则表达式：词法单元的**规约**
  - 有穷自动机：词法分析的**实现**
    - 正则表达式→有穷自动机→词法分析程序

- 有穷自动机分为两类：
  - 不确定的有穷自动机（Nondeterministic Finite Automata, **NFA**）
  - 确定的有穷自动机（Deterministic Finite Automata, **DFA**）

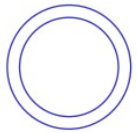
不确定的有穷自动机（NFA）

- 一个**不确定的有穷自动机（NFA）**包含：
  - 一个输入字母表 Σ
  - 一个有穷的状态集合 S
  - 一个开始状态或初始状态 s_0
  - 一个接受状态或终止状态集合 $F \subseteq S$
  - 一系列由某状态转换到后继状态的转换函数 $\text{state} \xrightarrow{\text{input}} \text{next state}$

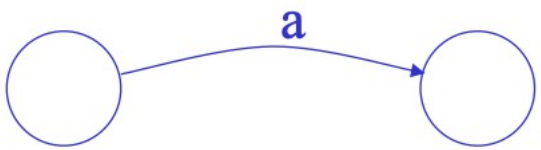
一个状态



一个接受状态

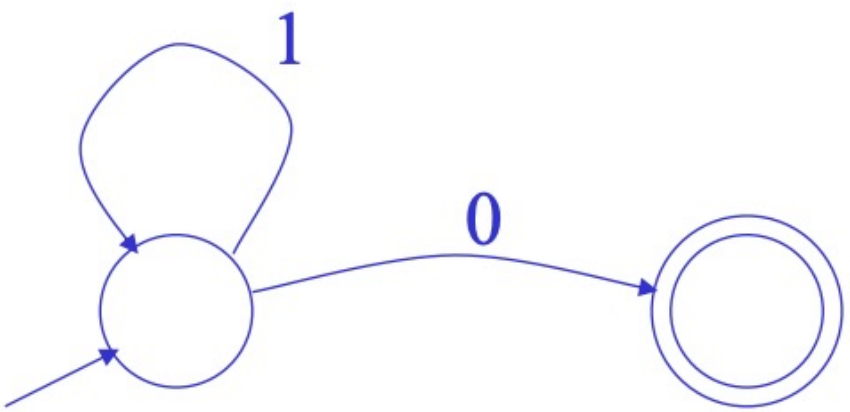


一个转换函数



NFA示例-1

- 字母表：二进制字母表{0,1}
- 一个NFA的转换图，描述了所有末尾跟一个0的任意多个1组成的数字序列



有穷自动机中输入字符串的接受

- 转换函数

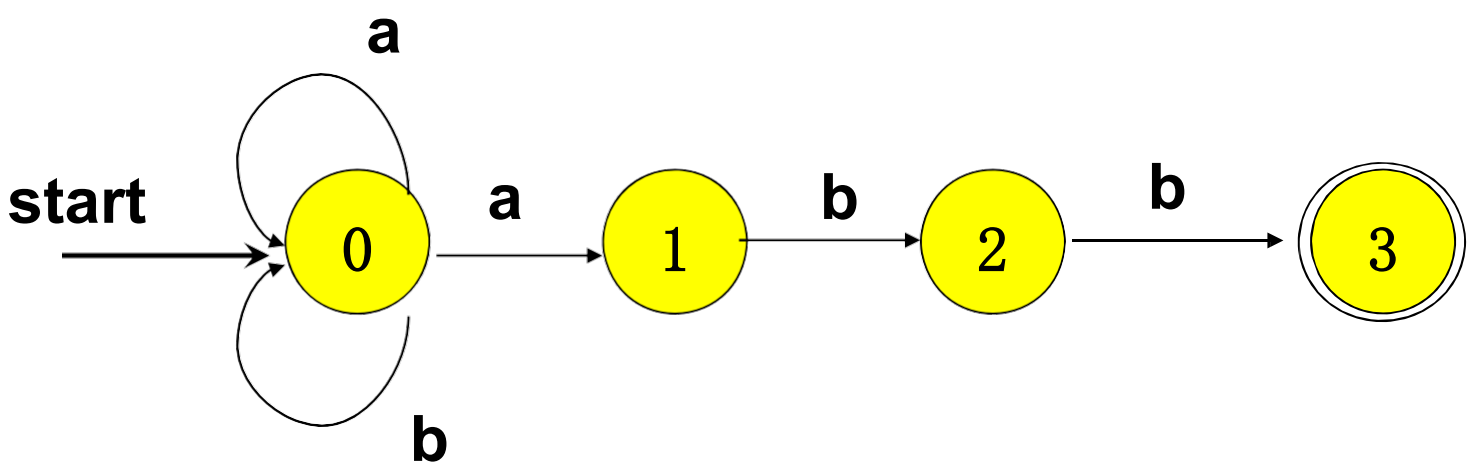
$$\textbf{state} \xrightarrow{\text{input}} \textbf{next state}$$

- 我们称一个有穷自动机**接受（accept）**一个输入字符串x，当且仅当对应的转换图中存在一条从开始状态到某个接受状态的路径，使得该路径上的各条边的标号组成符号串x
  - 必须是由**开始状态**到**接受状态**

- 由有穷自动机A**定义或接受**的语言L(A)，是从开始状态到某个接受状态的所有路径上的标号串的集合

NFA示例-2

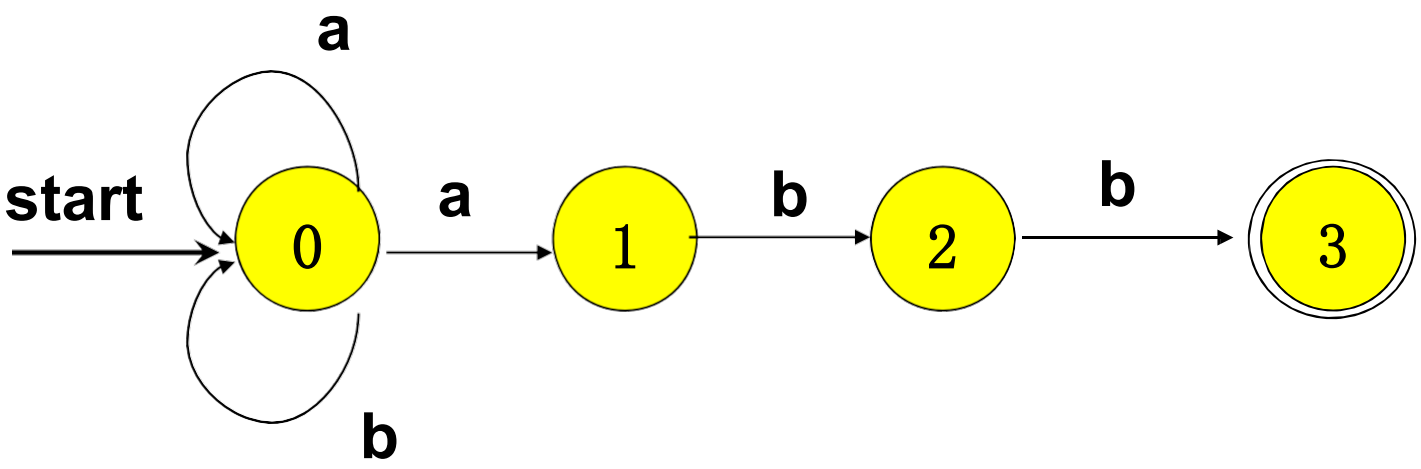
- 一个NFA，其接受所有由a和b组成的、以abb结尾的字符串



NFA 表示的语言：**(a|b)*abb**

NFA示例-2：转换表

- NFA的转换图



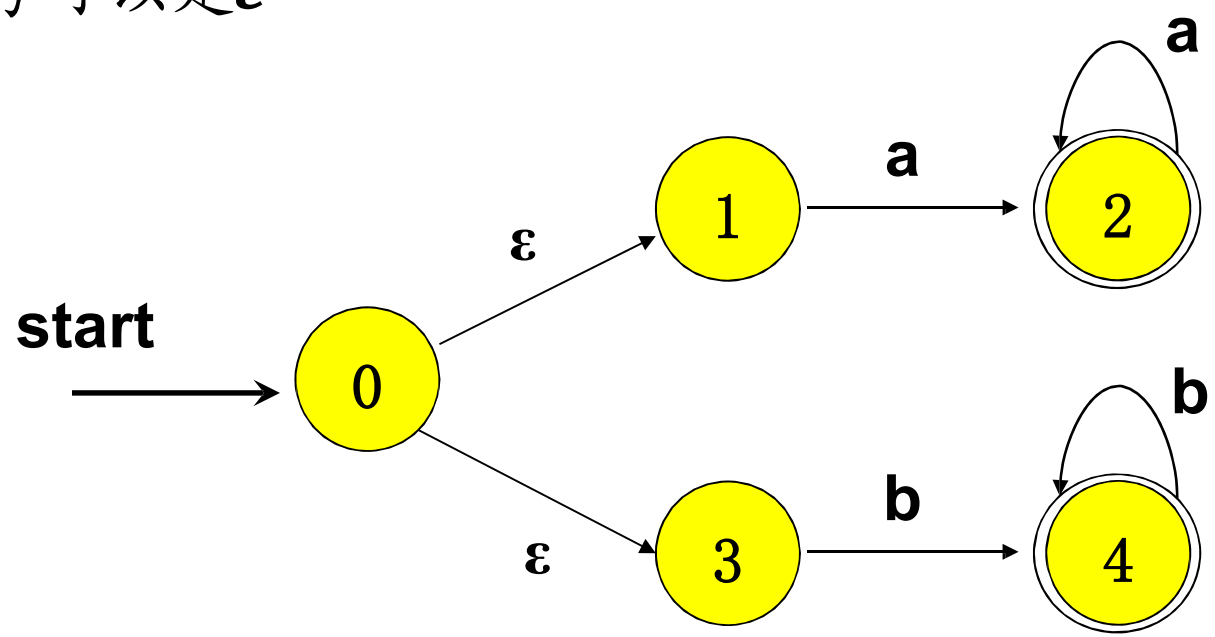
- 相应的转换表（transition table）
  - 各行对应于状态
  - 各列对应于输入符号和 ε
  - 内容为转换后的状态集合，空集代表没有对应转换

STATE	a	b	ϵ
0	$\{0,1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\emptyset$
2	$\emptyset$	$\{3\}$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

没有对应转换时占用空间大

NFA示例-3

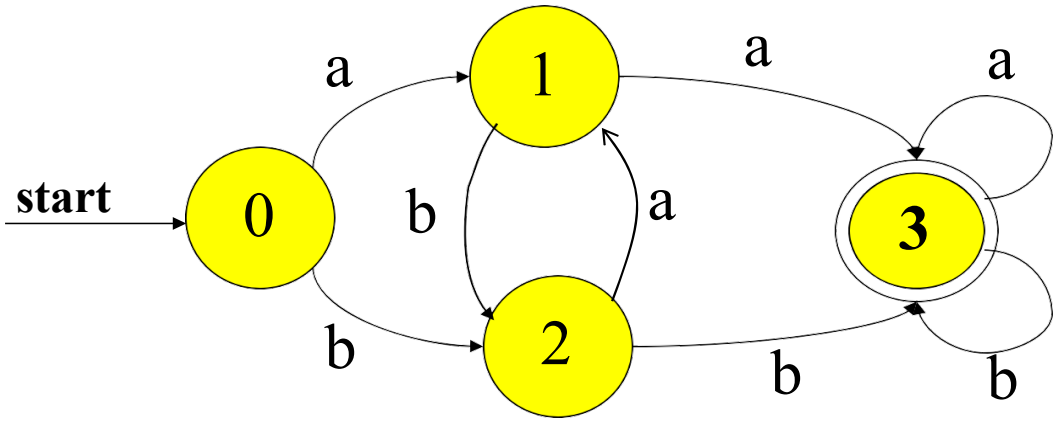
- 一个NFA，接受由**aa*|bb***定义的语言
  - 边的标号可以是**ε**



NFA 表示的语言：**aa*|bb***

确定的有穷自动机（DFA）

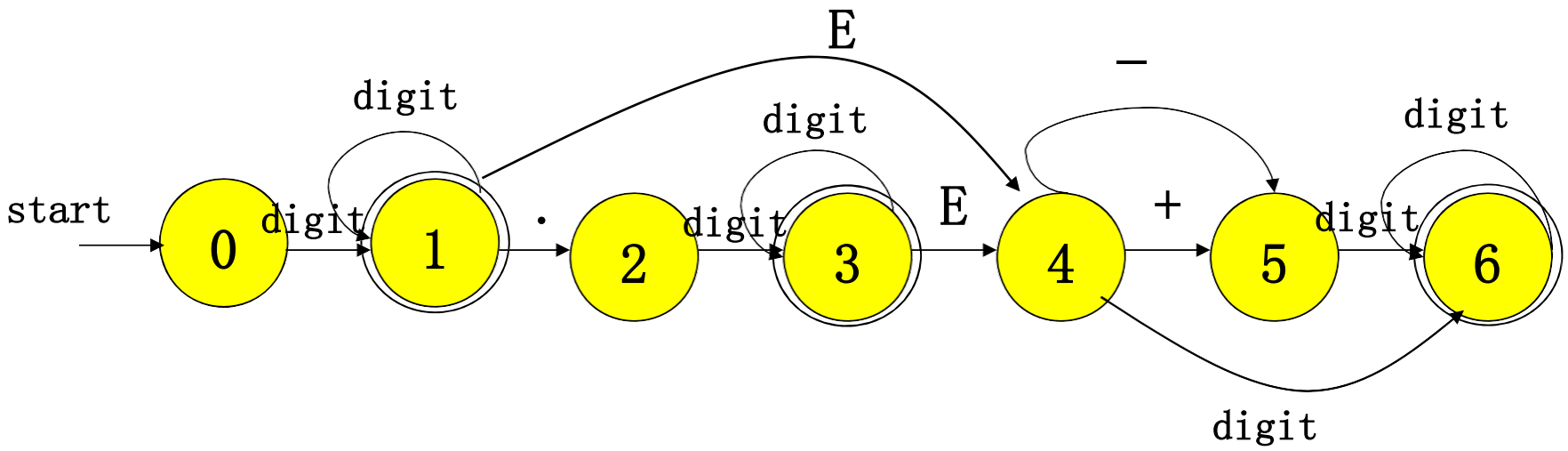
- 一个确定的有穷自动机（DFA）是NFA的特例：
  - 没有输入 **ε** 之上的转换动作
  - 对每个状态 **s** 和每个输入符号 **a**，有且只有一条标号为 **a** 的边离开
    - 如果用转换表来表示DFA，其每个表项就是一个确定的状态，而不是状态集合



状态	a	b
0	1	2
1	3	2
2	1	3
3	3	3

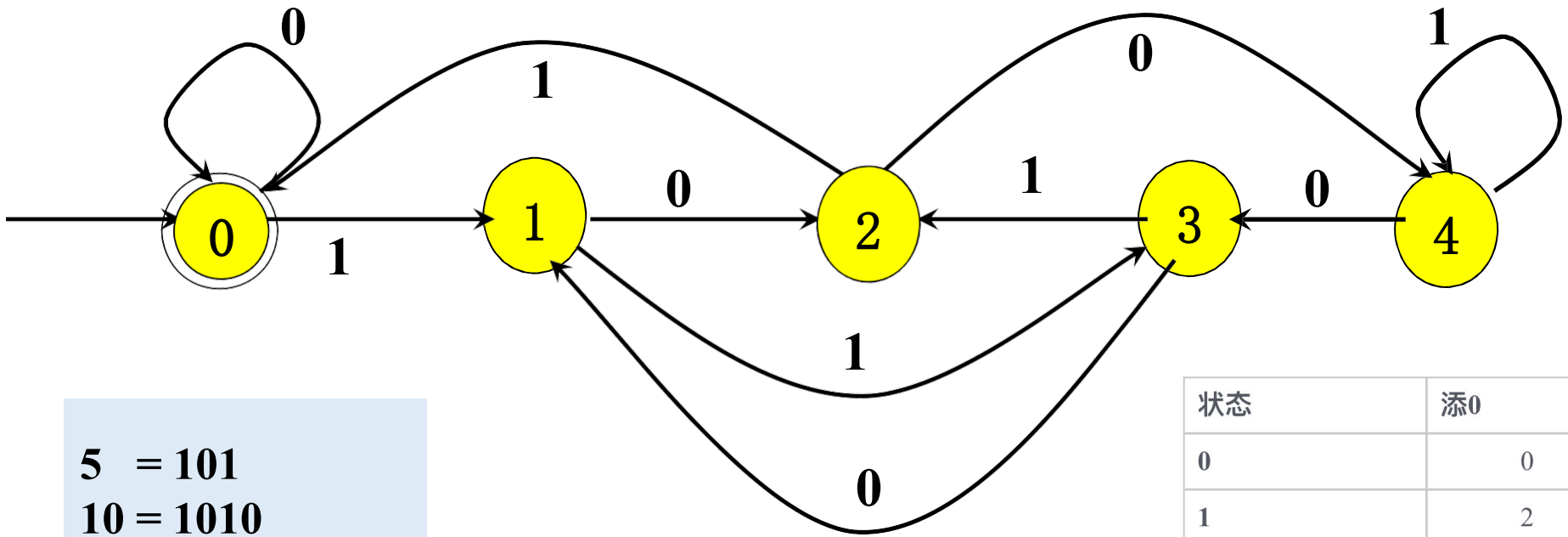
DFA示例-1

- 无符号数字的DFA



DFA示例-2

- 给出DFA，用于识别字母表Σ ={0,1}上能被5整除的二进制数



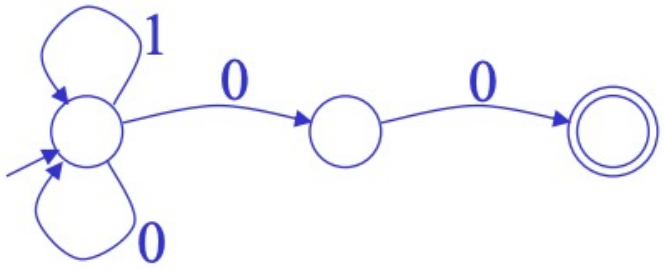
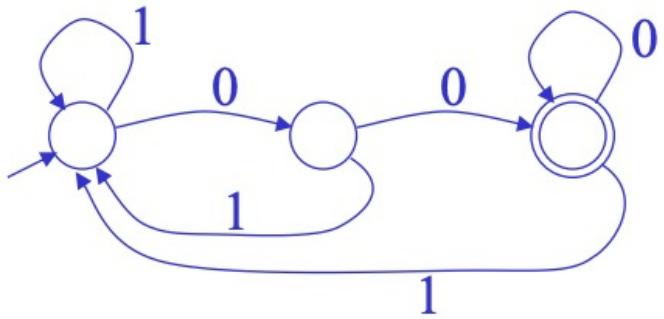
5 = 101
10 = 1010
15 = 1111
65 = 1000001

状态	添0	添1
0	0	1
1	2	3
2	4	0
3	1	2
4	3	4

NFA vs. DFA

- NFA和DFA的表达能力是等价的
  - 可以识别同一种语言
- DFA执行速度更快
  - 状态转换是确定的，不需要考虑其他选择
- NFA表示起来更简单、转换图更小

NFA



DFA